12/10/26, Revised from Claude.ai

Outline:

- Standard input, output, error
  - ทบทวนสั้นๆ: สามช่องทางของทุก process
  - 1. stdout: redirect ผลลัพธ์
  - 2. stderr: redirect เฉพาะ error
  - 3. รวมและทิ้ง
  - 4. stdin: ป้อนข้อมูลเข้าโปรแกรม
  - 5. `tee`: แยกสายท่อ
  - 6. ใช้ stdin/stdout/stderr ใน shell script
  - 7. ใน Python ก็ใช้หลักเดียวกัน
  - แบบฝึกหัด

Standard input, output, error

เรื่องนี้เราแตะไปแล้วนิดหนึ่งตอนเรียน file descriptor และ `2>&1` คราวนี้มาเรียนให้ครบ ทั้ง stdin, stdout, stderr และวิธีใช้งานจริงในทุกรูปแบบ

ทบทวนสั้นๆ: สามช่องทางของทุก process

FD	ชื่อ	ค่า default	ทิศทาง
`0`	**stdin**	คีย์บอร์ด	ข้อมูลเข้า
`1`	**stdout**	หน้าจอ	ผลลัพธ์ปกติออก
`2`	**stderr**	หน้าจอ	ข้อความ error ออก

แนวคิดสำคัญคือโปรแกรมไม่รู้เลยว่าข้อมูลมาจากไหนหรือไปไหน มันแค่อ่านจาก FD 0 และเขียนลง FD 1 กับ 2 ส่วน shell เป็นคน "เดินท่อ" ให้ว่าแต่ละช่องไปต่อกับอะไร (คีย์บอร์ด, ไฟล์, หรืออีกโปรแกรมหนึ่ง) นี่คือเหตุผลที่ `grep` ใช้ได้ทั้งกับไฟล์และกับ pipe โดยไม่ต้องแก้โค้ดสักบรรทัด

1. stdout: redirect ผลลัพธ์

```
echo "line one" > out.txt      # เขียนทับ
echo "line two" >> out.txt     # ต่อท้าย
cat out.txt
```

2. stderr: redirect เฉพาะ error

ลองสร้างสถานการณ์ที่มีทั้งผลลัพธ์ปกติและ error:

```
ls myfile.txt nothing_here.txt
```

จากนั้นแยกท่อ:

```
ls myfile.txt nothing_here.txt > good.txt 2> bad.txt
cat good.txt
cat bad.txt
```

คำสั่งเดียว แต่ผลลัพธ์แยกเป็นสองไฟล์ เพราะ `>` จัดการ FD 1 และ `2>` จัดการ FD 2

3. รวมและทิ้ง

Syntax	ความหมาย
`cmd > f 2>&1`	stdout และ stderr ลงไฟล์เดียวกัน
`cmd &> f`	เขียนสั้นแบบเดียวกัน (bash เท่านั้น)
`cmd 2> /dev/null`	ทิ้ง error ไม่ให้รกจอ
`cmd > /dev/null`	ทิ้งผลลัพธ์ปกติ เหลือแต่ error
`cmd > /dev/null 2>&1`	เงียบสนิท ไม่เห็นอะไรเลย

ตัวอย่างที่ใช้จริงบ่อย คือค้นไฟล์ทั้งระบบโดยไม่อยากเห็น "Permission denied" รกจอ:

```
find /etc -name "*.conf" 2> /dev/null | head
```

สังเกตว่า pipe (`|`) ส่งต่อเฉพาะ **stdout** เท่านั้น ส่วน stderr ยังหลุดขึ้นจอตามปกติ ถ้าอยากส่ง stderr เข้า pipe ด้วย ต้องใช้ `2>&1 |` หรือ `|&`

4. stdin: ป้อนข้อมูลเข้าโปรแกรม

stdin มีสามวิธีหลัก

จากไฟล์ด้วย `<`

```
wc -l < myfile.txt
```

สังเกตความต่างจาก `wc -l myfile.txt`: แบบแรก `wc` ไม่รู้ชื่อไฟล์เลย (อ่านจาก stdin) เลยแสดงแค่ตัวเลขอย่างเดียว ส่วนแบบหลังมันเปิดไฟล์เองจึงแสดงชื่อไฟล์ต่อท้ายด้วย (เราใช้เทคนิคนี้ใน `count_lines.sh` ไปแล้วเพื่อให้ได้ตัวเลขล้วนๆ)

จาก pipe ด้วย `|` (stdout ของโปรแกรมหนึ่งกลายเป็น stdin ของอีกโปรแกรม)

```
echo "hello world" | wc -w
```

จาก heredoc/here-string ด้วย `<<` และ `<<<`

```
cat << 'EOF'
บรรทัดนี้เป็น stdin ของ cat
EOF

grep hello <<< "hello world"
```

`<<<` (here-string) ป้อนข้อความบรรทัดเดียวเข้า stdin ตรงๆ สะดวกกว่า `echo ... |` เล็กน้อย

ลองใช้ stdin จากคีย์บอร์ดตรงๆ

```
cat
```

พิมพ์อะไรก็ได้แล้วกด Enter จะเห็น `cat` สะท้อนกลับมาทีละบรรทัด กด `Ctrl+D` เพื่อบอกว่า "จบ input แล้ว" (ส่ง EOF) นี่คือ `cat` ที่ไม่มี argument จึงอ่านจาก stdin ซึ่งคือคีย์บอร์ดนั่นเอง

5. `tee`: แยกสายท่อ

จำได้ไหมครับว่าเราเจอ `tee` ตอนแก้ปัญหา `sudo echo >` มันอ่านจาก stdin แล้วเขียนทั้งลงไฟล์และส่งต่อไป stdout:

```
ls -l | tee listing.txt | wc -l
```

ข้อมูลไหลผ่าน `tee` ไปสองทาง: ลงไฟล์ `listing.txt` และต่อไปให้ `wc`

6. ใช้ stdin/stdout/stderr ใน shell script

อ่านทีละบรรทัดจาก stdin:

```
#!/bin/bash
while read -r line; do
    echo "ได้รับ: $line"
done
```

รันแบบนี้ได้หลายทาง:

```
./reader.sh < myfile.txt
```

```
ls | ./reader.sh
```

script เดียวกัน อ่านได้ทั้งจากไฟล์และจาก pipe เพราะมันอ่านจาก stdin เสมอ

เขียน error ลง stderr อย่างถูกต้อง:

```
#!/bin/bash
if [ -z "$1" ]; then
    echo "Error: ต้องระบุชื่อไฟล์" >&2
    exit 1
fi
echo "กำลังประมวลผล $1"
```

`>&2` คือส่ง stdout ของ `echo` ไปยัง FD 2 ข้อดีคือถ้ามีคน redirect `> result.txt` ข้อความ error จะไม่ไปปนในไฟล์ผลลัพธ์ นี่คือเหตุผลที่ `log_error` ใน script ติดตั้งก่อนหน้านี้ใช้ `>&2`

7. ใน Python ก็ใช้หลักเดียวกัน

```
#!/usr/bin/env python3
import sys

for line in sys.stdin:
    print(f"ได้รับ: {line.strip()}")

print("เกิดข้อผิดพลาด", file=sys.stderr)
```

`sys.stdin`, `sys.stdout`, `sys.stderr` คือ FD 0, 1, 2 ตัวเดียวกัน ดังนั้น script Python ต่อ pipe กับคำสั่ง Linux ได้เหมือน `grep` หรือ `wc` ทุกประการ

แบบฝึกหัด

ลองทำแล้วส่งผลลัพธ์มาดูครับ (ทายผลก่อนรัน):

1. รัน `ls myfile.txt nothing_here.txt > good.txt 2> bad.txt` แล้ว `cat` ทั้งสองไฟล์ ไฟล์ไหนมีอะไร
2. รัน `ls myfile.txt nothing_here.txt 2>&1 | wc -l` กับ `ls myfile.txt nothing_here.txt | wc -l` ตัวเลขต่างกันไหม ทำไม
3. รัน `ls myfile.txt nothing_here.txt 2>&1 > both.txt` แล้ว `cat both.txt` ผลตรงกับที่คาดไหม (คำใบ้: เรื่องลำดับของ `2>&1` ที่เรียนไปแล้ว)
4. เขียน `reader.sh` ตามตัวอย่างข้างบน แล้วลองป้อนข้อมูลสามแบบ: `< myfile.txt`, `ls | ./reader.sh` และพิมพ์เองจากคีย์บอร์ด (จบด้วย `Ctrl+D`)